FEST Summer 2026 — Student Management System

Documentation of `main.py`

---

1. Background

This project is a desktop application built for the **FEST Summer 2026** program at Vanderbilt University. It is a single-file Python program (`main.py`) that gives an instructor or admin a way to manage a small student academic roster — adding, editing, and deleting enrollment records, computing letter grades and GPA, visualizing class performance, and generating a Vanderbilt-styled student ID card — all through a desktop GUI (no web server, no database server; everything is stored in local files sitting next to the script).

The program is built with **Tkinter** (Python's built-in GUI toolkit) for the interface, **Matplotlib** (optionally boosted by **Seaborn**/**Pandas**) for charts, and plain **CSV/JSON** files for storage. It has no external services and no installation beyond the Python packages it imports, which makes it easy to run on a lab machine or a personal laptop.

Who uses it

- A login-protected user (e.g., an instructor or TA) registers an account and logs in.
- Once logged in, they land on a **Main Menu** with 10 numbered options (0–9), each opening a popup window that does one job (add a record, chart grades, print an ID card, run canned queries, etc.).
- The underlying student roster (`students.csv`) is shared across all options — nothing is scoped to the logged-in account. The login system in this app protects *access to the tool*, not *ownership of individual student records*.

---

2. Data & File Layout

All data files live in the same folder as `main.py` (computed once at import time via `BASE_DIR = os.path.dirname(os.path.abspath(__file__))`):

File	Format	Purpose
`users.csv`	CSV, columns `id,password`	App login accounts. Passwords are stored as SHA-256 hashes, never in plain text.
`students.csv`	CSV, columns `student_id,name,major,course_code,course_title,points,gpa`	The academic roster. **One row per (student, course) enrollment** — a student with 4 courses has 4 rows sharing the same `student_id`.
`gpa_scale.json`	JSON array of `[min, max, letter, gpa]` rows	The editable points→letter→GPA grading scale (Option 3 lets a user rewrite this at runtime).

Because a student can appear in multiple rows, almost every screen that needs "one entry per student" (Student Profile, Student ID Card, Queries) first re-groups the flat CSV rows into a dictionary keyed by `student_id`.

---

3. Functionalities (Main Menu Options)

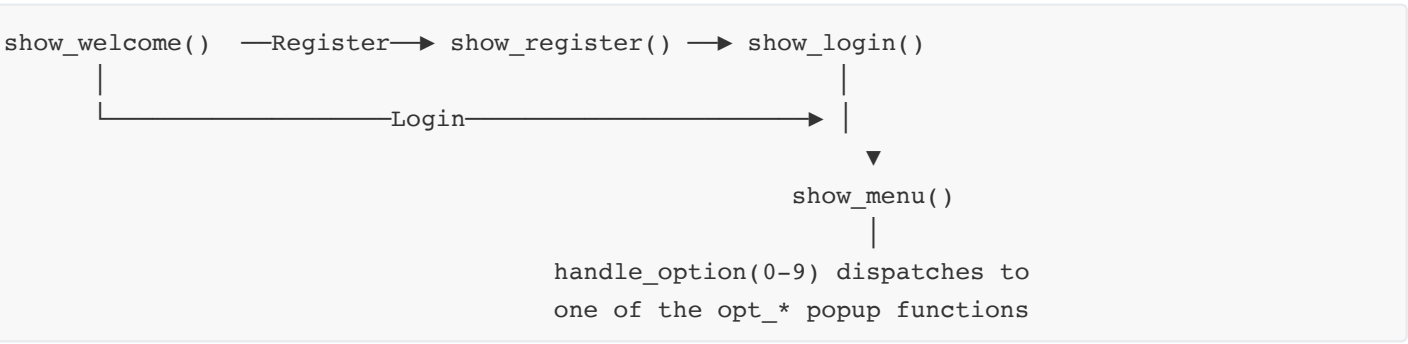
After logging in, the Main Menu offers:

#	Label	What it does
0	Add Student	Form to append a new (student, course) enrollment row to `students.csv`, auto-computing the letter grade/GPA as you type.
1	Edit Student	Table of all rows; select one, edit its fields in a popup, save back to the CSV.
2	Delete Record	Table of all rows with multi-select; permanently removes the selected row(s) after confirmation.
3	GPA Formula	View/edit the points-range → letter-grade → GPA-value scale used everywhere else in the app; persisted to `gpa_scale.json`; resettable to defaults.
4	Swarm Plot	Matplotlib/Seaborn swarm plot of every student's score, grouped by course code and colored by grade band.
5	Major Count Plot	Bar chart of how many enrollment rows belong to each major.
6	Grade Pie Chart	Pie chart of the letter-grade distribution (A/B/C/D/F) with a fake-3D beveled look and the single largest slice pulled out.
7	Student Profile	Browsable, searchable single-student view: identity card + a table of that student's courses/grades, with Previous/Next navigation.
8	Student ID Card	A Vanderbilt-styled mock student ID card (procedurally generated cartoon avatar, barcode, gold trim) with Previous/Next/Search navigation.
9	Queries	A fixed panel of 10 canned analytical questions (e.g., "How many CS students got an A+?") computed live from the current roster.

---

4. Architecture & Navigation Flow

The whole UI lives in **one Tkinter root window** (`root`). Instead of opening a new window for every screen, top-level "pages" (Welcome → Register/Login → Main Menu) are drawn directly into `root` and each new page starts by wiping out whatever was drawn before it (`clear_screen()`). Menu **options** (0–9), on the other hand, open as separate modal `Toplevel` popup windows layered on top of the menu, so the user can close a tool and land right back on the menu without it having been rebuilt.



State that must survive screen changes is kept in a couple of module-level globals: `root` (the Tk window) and `current_user` (the logged-in account id). Everything else (GPA scale, student list) is read fresh from disk each time a screen needs it, so edits made in one popup are immediately visible the next time another popup opens.

A note on "line-by-line" in this document

Tkinter screens in this file are built from long runs of near-identical `tk.Frame(...).pack(...)` / `make_label(...).pack(...)` calls (the "colored border → dark inner card → canvas button" pattern is repeated dozens of times). Walking through every one of those individually would make this document unreadable without adding real understanding. So below: **helper functions and anything with actual logic (grades, GPA math, CSV I/O, query predicates, dispatch tables) are explained line by line**; **repetitive widget-layout code is explained in short grouped blocks**, with the pattern named once so you can recognize it everywhere else it appears.

---

5. Imports & Global Configuration

```
import tkinter as tk
from tkinter import messagebox, ttk
import os, hashlib, json
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
import numpy as np
import pandas as pd

try:
    import seaborn as sns
    HAS_SEABORN = True
except ImportError:
    HAS_SEABORN = False
```

- `tkinter as tk` — the GUI toolkit; every window, button, and label comes from here.
- `messagebox, ttk` — `messagebox` gives the pop-up alert/confirm dialogs (`showerror`, `showinfo`, `askyesno`); `ttk` is Tkinter's "themed" widget set, used here for `Treeview` (tables), `Scrollbar`, `Style`, and (after a later fix) `Button`.
- `os, hashlib, json` — standard library: `os` builds file paths and checks existence, `hashlib` hashes passwords, `json` persists the editable GPA scale.
- `matplotlib.pyplot as plt` and `backend_tkagg` — draw charts (swarm plot, bar chart, pie chart) and embed the resulting `Figure` directly inside a Tkinter window instead of a separate plot window.
- `numpy as np` — used for the random-but-deterministic jitter in the swarm plot fallback and for seeding the ID card's barcode pattern.
- `pandas as pd` — a **required** import (not optional). All roster/account reading and writing (`load_students`, `save_students`, `append_student`, `load_users`, `save_user` — see §7 and §9) goes through pandas `DataFrame`s and `read_csv`/`to_csv` instead of the standard-library `csv` module the project originally used. Because core I/O now depends on it, the project's virtual environment must have `pandas` installed (`pip install pandas`), or the app will fail to start.
- The `try/except` around `seaborn` makes it (and only it) **optional**: if it isn't installed, `HAS_SEABORN` becomes `False` and the two chart functions (Swarm Plot, Major Count Plot) fall back to plain Matplotlib/NumPy implementations instead of crashing. Note `pandas` itself is *not* part of this optional block — the two chart functions still build a `pd.DataFrame` for Seaborn to consume when `HAS_SEABORN` is `True`, but the app's file I/O uses pandas unconditionally either way.

```
BASE_DIR       = os.path.dirname(os.path.abspath(__file__))
USERS_FILE     = os.path.join(BASE_DIR, "users.csv")
STUDENTS_FILE  = os.path.join(BASE_DIR, "students.csv")
GPA_SCALE_FILE = os.path.join(BASE_DIR, "gpa_scale.json")
STUDENT_FIELDS = ["student_id", "name", "major", "course_code", "course_title", "points",
"gpa"]
```

- `BASE_DIR` — the absolute folder containing `main.py`, computed once so the data files are found no matter what directory the script is *launched* from.
- `USERS_FILE`, `STUDENTS_FILE`, `GPA_SCALE_FILE` — full paths to the three data files described in §2.
- `STUDENT_FIELDS` — the canonical CSV column order for the roster; every write to `students.csv` uses this list so the header/row order never drifts.

```
BG     = "#0d0d1a"
PANEL  = "#16213e"
CARD   = "#1a1a35"
ACCENT = "#7c3aed"
CYAN   = "#22d3ee"
TEAL   = "#0d7377"
TEXT   = "#f1f5f9"
MUTED  = "#94a3b8"
GREEN  = "#10b981"
RED    = "#ef4444"
BORDER = "#2a2a4a"
```

A single dark-mode color palette (background, panel, card, accent, text, etc.) used everywhere so the whole app looks visually consistent without repeating hex codes.

```
OPTION_COLORS = [
    "#7c3aed", "#0d7377", "#b45309", "#1d4ed8", "#be185d", "#065f46",
    "#9d174d", "#0e7490", "#1e3a5f", "#15803d",
]
OPTION_LABELS = [
    "Add Student", "Edit Student", "Delete Record",
    "GPA Formula", "Swarm Plot",  "Major Count Plot",
    "Grade Pie Chart", "Student Profile", "Student ID Card",
    "Queries",
]
```

Parallel lists indexed 0–9: `OPTION_COLORS[i]` is the accent color and `OPTION_LABELS[i]` is the button text for menu option `i`. Keeping them as parallel lists (instead of, say, a list of dicts) is what lets the main-menu grid be built with a single `for i in range(10):` loop (see §11).

```
DEFAULT_GPA_SCALE = [
    (97, 100, "A+", 4.0), (93, 96, "A",  4.0), (90, 92, "A-", 3.7),
    (87,  89, "B+", 3.3), (83, 86, "B",  3.0), (80, 82, "B-", 2.7),
    (77,  79, "C+", 2.3), (73, 76, "C",  2.0), (70, 72, "C-", 1.7),
    (67,  69, "D+", 1.3), (63, 66, "D",  1.0), (60, 62, "D-", 0.7),
    ( 0,  59, "F",  0.0),
]
GPA_SCALE = list(DEFAULT_GPA_SCALE)
```

`DEFAULT_GPA_SCALE` is the factory grading scale: each tuple is `(min_points, max_points, letter, gpa_value)`. `GPA_SCALE` is a **mutable copy** of it — the copy is what Option 3 edits and what `points_to_grade()` actually reads, while `DEFAULT_GPA_SCALE` stays untouched so "Reset Defaults" always has an original to restore from.

```
GRADE_PALETTE = {
    "A (90-100)": "#10b981",
    "B (80-89)":  "#3b82f6",
    "C (70-79)":  "#f59e0b",
    "D (60-69)":  "#f97316",
    "F (0-59)":   "#ef4444",
}
```

Maps the coarse (letter-band, not exact `A+/A-`) grade category to a chart color, used by the swarm plot's legend/coloring and its horizontal threshold lines.

```
root         = None
current_user = None
```

The two pieces of state that must be visible from *every* function without being passed as parameters: `root` is set once in `main()` to the actual Tk window; `current_user` is set on successful login and cleared on logout. Functions that assign to them use `global root` / `global current_user` (see `main()` and `do_logout()`).

---

6. GPA Helpers

```
def points_to_grade(pts):
    p = max(0, min(100, round(float(pts))))
    for lo, hi, letter, gpa in GPA_SCALE:
        if lo <= p <= hi:
            return letter, gpa
    return "F", 0.0
```

Converts a raw numeric score into a `(letter, gpa)` pair using the current `GPA_SCALE`.

- `float(pts)` — accepts either a string (from an `Entry` widget) or a number.
- `round(...)` — scores like `89.6` round to `90` before banding, so a student is not penalized for a value that would visually read as an A-.
- `max(0, min(100, ...))` — clamps the rounded score into `[0, 100]` so an out-of-range value (which shouldn't normally occur, since forms validate `0–100`) can never fail to match a band.
- The `for` loop scans `GPA_SCALE` (a list of `(lo, hi, letter, gpa)` tuples) and returns the first band whose `[lo, hi]` contains `p`.
- `return "F", 0.0` — a safety net: if the scale has gaps (possible after a user edits it in Option 3) and no band matches, the function still returns a valid pair instead of raising.

```
def grade_category(p):
    p = float(p)
    if p >= 90: return "A (90-100)"
    if p >= 80: return "B (80-89)"
    if p >= 70: return "C (70-79)"
    if p >= 60: return "D (60-69)"
    return "F (0-59)"
```

A second, simpler classifier used only for **charting** (the swarm plot). Unlike `points_to_grade`, it doesn't consult the editable `GPA_SCALE` — it always uses fixed 90/80/70/60 cutoffs, because its output must match the fixed keys of `GRADE_PALETTE` above. Each `if` returns immediately, so it behaves like a first-match cascade from high to low.

---

7. CSV Helpers — Users (Login Accounts)

```
def hash_password(pw):
    return hashlib.sha256(pw.encode()).hexdigest()
```

Turns a plaintext password into its SHA-256 hex digest. `pw.encode()` converts the Python string to bytes (required by `hashlib`); `.hexdigest()` returns the hash as a readable hex string, which is what actually gets stored/compared — the plaintext password itself is never written to disk.

```
def load_users():
    if not os.path.exists(USERS_FILE):
        return {}
    try:
        # dtype=str + keep_default_na=False: keep every value as plain text,
        # exactly like csv.DictReader used to (otherwise pandas would try to
        # infer numeric ids and turn blank fields into NaN).
        df = pd.read_csv(USERS_FILE, dtype=str, keep_default_na=False)
    except pd.errors.EmptyDataError:
        return {}
    return dict(zip(df["id"], df["password"]))
```

- If `users.csv` doesn't exist yet (fresh install, nobody has registered), returns an empty dict rather than erroring.
- Otherwise reads the whole file in one call with `pd.read_csv`, which by default would try to *infer* a type per column (turning e.g. an all-numeric `id` column into `int64`, and any blank cell into `NaN`). `dtype=str` forces every column to stay plain Python strings, and `keep_default_na=False` stops pandas from converting empty cells to `NaN` — together they reproduce exactly what `csv.DictReader` used to hand back, so nothing downstream that expects string ids/passwords breaks.
- Reading a file that exists but has **only a header row** (0 data rows) raises `pandas.errors.EmptyDataError` in some pandas versions for a truly empty file; the `try/except`

catches that and returns `{}` the same as the "file doesn't exist" branch.

- `dict(zip(df["id"], df["password"]))` zips the `id` column and `password` column together into a single `{user_id: hashed_password}` lookup table — the pandas equivalent of the old dict comprehension over `csv.DictReader` rows.

```
def save_user(uid, hpw):
    exists = os.path.exists(USERS_FILE)
    row = pd.DataFrame([{"id": uid, "password": hpw}])
    row.to_csv(USERS_FILE, mode="a", header=not exists, index=False)
```

Appends one new account row.

- `exists = os.path.exists(...)` is checked **before** opening/writing the file, for the same reason as before: once you've opened the file in append mode, the file always "exists," so the check has to happen first.
- `pd.DataFrame([{"id": uid, "password": hpw}])` builds a tiny, one-row DataFrame out of the new account.
- `row.to_csv(USERS_FILE, mode="a", header=not exists, index=False)` appends that single row straight onto the CSV: `mode="a"` opens in append mode (existing accounts untouched), `header=not exists` writes the `id,password` header line only the first time the file is created, and `index=False` stops pandas from writing its own `0,1,2,…` row-number column (which `csv.DictWriter` never had, so this keeps the file format identical to before).

---

8. GPA Scale Persistence

```
def load_gpa_scale():
    global GPA_SCALE
    if os.path.exists(GPA_SCALE_FILE):
        try:
            with open(GPA_SCALE_FILE) as f:
                data = json.load(f)
            GPA_SCALE = [tuple(row) for row in data]
        except Exception:
            GPA_SCALE = list(DEFAULT_GPA_SCALE)
```

Called once at startup (`main()`).

- `global GPA_SCALE` — needed because the function reassigns the module-level `GPA_SCALE` variable rather than just reading it.
- If `gpa_scale.json` exists, it's parsed with `json.load`, and each row (a JSON array `[lo, hi, letter, gpa]`) is converted to a Python `tuple` so it matches the shape `points_to_grade` expects.
- The broad `except Exception` means a corrupted or hand-edited JSON file can't crash the app on

startup — it just silently falls back to `DEFAULT_GPA_SCALE`.

- If the file doesn't exist at all (first run), `GPA_SCALE` is simply left as whatever it was initialized to at import time (`list(DEFAULT_GPA_SCALE)`), so nothing needs to happen in that branch.

```
def save_gpa_scale():
    with open(GPA_SCALE_FILE, "w") as f:
        json.dump(GPA_SCALE, f, indent=2)
```

Writes the current in-memory `GPA_SCALE` back out as pretty-printed JSON (`indent=2`) every time it changes (called from `save_row()` and `reset_defaults()` in Option 3).

---

9. CSV Helpers — Students (the Roster)

```
def load_students():
    if not os.path.exists(STUDENTS_FILE):
        return []
    try:
        # dtype=str + keep_default_na=False keeps student_id/points/gpa as
        # plain strings (never int64/float64/NaN), so every comparison and
        # float(...)/f"{...}" call elsewhere in the app keeps working exactly
        # as it did when the file was read with csv.DictReader.
        df = pd.read_csv(STUDENTS_FILE, dtype=str, keep_default_na=False)
    except pd.errors.EmptyDataError:
        return []
    return df.to_dict("records")
```

Reads the entire roster into memory as a **list of dicts** (one dict per CSV row, keys = `STUDENT_FIELDS`) — same shape every other function in the file expects, just produced by pandas instead of the standard-library `csv` module.

- `pd.read_csv(STUDENTS_FILE, dtype=str, keep_default_na=False)` reads the whole file at once. This is the same `dtype=str` / `keep_default_na=False` combo used in `load_users()` (§7), and it matters even more here: `student_id` values like `100`–`199` look purely numeric, so without `dtype=str` pandas would silently turn the column into `int64` — which would break every place in the app that compares a student ID against text typed into a search box (`sid in student_dict`, `student_ids.index(sid)`, etc.), since a search box always returns a `str`.
- `except pd.errors.EmptyDataError: return []` — a header-only `students.csv` (e.g. right after every row has been deleted) has zero data rows; some pandas versions raise this error for a file with no parsable rows at all, so it's caught and treated the same as "no file yet."
- `df.to_dict("records")` converts the DataFrame into the list-of-dicts shape (`[{"student_id": "100", "name": "...", ...}, ...]`) that every `opt_*` function already expects — this is the one line that makes the pandas rewrite a drop-in replacement for the old `list(csv.DictReader(f))`, with no changes needed anywhere else in the file that calls `load_students()`.

Every option that touches student data starts by calling this — there is no caching, so it always reflects the latest saved state on disk.

```
def save_students(records):
    df = pd.DataFrame(records, columns=STUDENT_FIELDS)
    df.to_csv(STUDENTS_FILE, index=False)
```

Overwrites `students.csv` from scratch with whatever `records` list is passed in. `pd.DataFrame(records, columns=STUDENT_FIELDS)` builds a DataFrame from the list of dicts, explicitly pinning the column order to `STUDENT_FIELDS` (so column order can never drift even if a dict happens to have its keys in a different order); `df.to_csv(STUDENTS_FILE, index=False)` writes it back out, header first then every row, with `index=False` again suppressing pandas' own row-number column. Passing an **empty** list still produces a valid header-only CSV (since the DataFrame is built with `columns=STUDENT_FIELDS` even when it has zero rows) — this is what Option 2 (Delete) relies on when every remaining record is removed. Used after an edit (Option 1) or a delete (Option 2), where the whole in-memory list has already been mutated and just needs to be flushed back to disk.

```
def append_student(rec):
    exists = os.path.exists(STUDENTS_FILE)
    row = pd.DataFrame([rec], columns=STUDENT_FIELDS)
    row.to_csv(STUDENTS_FILE, mode="a", header=not exists, index=False)
```

Same append pattern as `save_user()` (§7), but for one new enrollment row (`rec` is a dict with the 7 `STUDENT_FIELDS` keys): wrap the single row in a one-row DataFrame (again pinning `columns=STUDENT_FIELDS` for consistent ordering), then append it with `mode="a"`, writing the header only if the file didn't already exist. Used by Option 0 (Add Student) so adding one student doesn't require rewriting the whole file.

> **Why pandas instead of the `csv` module?** The original version of this project used Python's built-in `csv.DictReader`/`csv.DictWriter` for all of this. The behavior is intentionally unchanged — same file format, same header, same append-vs-overwrite semantics, same plain-string field values — pandas is simply the engine doing the reading/writing now. This was verified by comparing pandas' output against the old `csv`-module output on the real 150-row roster: identical row count, identical field values, and identical Python types (`str`, not any pandas/numpy type) in every returned dict.

---

10. Widget Factory Functions

These are small builders that wrap raw Tkinter widget calls so every screen in the app looks and behaves the same without repeating boilerplate.

```
def make_label(parent, text, size=11, bold=False, color=None, **kw):
    return tk.Label(
        parent, text=text,
        font=("Segoe UI", size, "bold" if bold else "normal"),
        fg=color or TEXT, bg=parent.cget("bg"), **kw,
    )
```

Builds a themed `tk.Label`. `font=(..., "bold" if bold else "normal")` picks the font weight from the `bold` flag; `fg=color or TEXT` uses the passed-in color or falls back to the default text color; `bg=parent.cget("bg")` reads the **parent's** current background and reuses it, so the label always blends into whatever frame it's dropped into instead of needing the caller to specify a background every time. `**kw` forwards any extra Tkinter options (e.g. `wraplength`) straight through.

```
def make_entry(parent, show="", width=32):
    wrap = tk.Frame(parent, bg=CYAN, padx=1, pady=1)
    e = tk.Entry(
        wrap, font=("Segoe UI", 13), bg=CARD, fg=TEXT,
        insertbackground=CYAN, relief="flat", width=width, show=show,
    )
    e.pack()
    return wrap, e
```

Creates a text field with a thin cyan "glow" border: a 1-pixel-padded `Frame` (`wrap`) filled with `CYAN` sits *behind* the actual `Entry`, so only a 1px cyan outline peeks out around the dark entry box. `show=show` lets the caller pass `"●"` to mask password input. The function returns **both** the outer wrapper (which the caller `.pack()`s into the layout) and the inner `Entry` (which the caller reads `.get()` from) — that's why call sites do `wrap, entry = make_entry(...)`.

```
def make_button(parent, text, cmd, color=None, width=260, height=54, radius=26, fg="white"):
    """Canvas-based rounded-corner button."""
```

This is the app's signature rounded button, and it exists because plain `tk.Button` cannot draw rounded corners (and, as discovered while fixing the ID card page, ignores background color entirely on macOS). Instead it draws the button by hand on a `tk.Canvas`:

```
    color = color or ACCENT
    try:
        bg = parent.cget("bg")
    except Exception:
        bg = BG
    cv = tk.Canvas(parent, width=width, height=height,
                   bg=bg, highlightthickness=0, bd=0)
```

Picks the fill color (default `ACCENT`), reads the parent's background so the canvas's own corners (which stay square) blend in, and creates a borderless `Canvas` of the requested size.

```
    def _draw(hover=False):
        cv.delete("all")
        r = radius
        x1, y1, x2, y2 = 1, 1, width - 1, height - 1
        c = color
```

```
        cv.create_arc(x1,       y1,       x1+2*r, y1+2*r, start=90,  extent=90, fill=c,
outline=c)
        cv.create_arc(x2-2*r,   y1,       x2,     y1+2*r, start=0,   extent=90, fill=c,
outline=c)
        cv.create_arc(x1,       y2-2*r,   x1+2*r, y2,     start=180, extent=90, fill=c,
outline=c)
        cv.create_arc(x2-2*r,   y2-2*r,   x2,     y2,     start=270, extent=90, fill=c,
outline=c)
        cv.create_rectangle(x1+r, y1,    x2-r, y2,    fill=c, outline=c)
        cv.create_rectangle(x1,   y1+r,  x2,   y2-r,  fill=c, outline=c)
        if hover:
            cv.create_arc(x1+3,     y1+3,     x1+2*r, y1+2*r, start=90,  extent=90,
outline="#ffffff30", style="arc", width=2)
            cv.create_arc(x2-2*r,   y1+3,     x2-3,   y1+2*r, start=0,   extent=90,
outline="#ffffff30", style="arc", width=2)
        cv.create_text(width // 2, height // 2, text=text,
                       fill=fg, font=("Segoe UI", 13, "bold"))
    _draw()
```

`_draw()` is a nested function that renders the button's face from scratch every time it's called: `cv.delete("all")` clears the canvas, then four `create_arc` calls draw the rounded corners (one 90° arc per corner, matched to `radius`) and two `create_rectangle` calls fill the flat top/bottom and left/right strips so the arcs and rectangles together form one seamless rounded rectangle. If `hover=True`, two translucent white arcs are drawn on top of the top corners as a subtle highlight ring. Finally the button's `text` is drawn centered on top. Calling `_draw()` once immediately after defining it paints the button's resting state.

```
    def on_enter(e):
        cv.config(cursor="hand2")
        _draw(hover=True)
    def on_leave(e):
        cv.config(cursor="")
        _draw()
    def on_click(e):
        if cmd:
            cmd()
    cv.bind("<Enter>",    on_enter)
    cv.bind("<Leave>",    on_leave)
    cv.bind("<Button-1>", on_click)
    return cv
```

Three mouse-event handlers are bound to the canvas: `<Enter>` / `<Leave>` (mouse entering/leaving the widget) toggle the hand cursor and redraw with/without the hover ring; `<Button-1>` (left click) calls `cmd()` if one was supplied — this is how the button's "click behavior" is wired up, since the returned object is a `Canvas`, not a real `Button`. The function returns the canvas itself so the caller can `.pack()` / `.grid()` it like any other widget.

```
def make_link(parent, text, cmd):
    return tk.Button(
        parent, text=text, command=cmd,
        font=("Segoe UI", 11, "underline"),
        fg=CYAN, bg=parent.cget("bg"),
        activeforeground=CYAN, activebackground=parent.cget("bg"),
        relief="flat", cursor="hand2", bd=0,
    )
```

A lightweight "hyperlink"-styled button (underlined cyan text, no border/relief) used for small secondary actions like "Already registered? Login" — small enough that the macOS native-button color quirk doesn't matter here (there's no background to lose).

```
def make_top_bar(title, accent=None):
    bar = tk.Frame(root, bg=PANEL)
    bar.pack(fill="x")
    tk.Frame(bar, bg=accent or ACCENT, height=5).pack(fill="x")
    make_label(bar, title, 18, bold=True).pack(pady=16)
    return bar
```

Builds the thin colored accent strip + bold title header used at the top of the Register and Login screens. `accent or ACCENT` lets each screen pick its own strip color (e.g. cyan for Login) while defaulting to the app's main accent purple.

```
def make_popup(title, w=600, h=520, resizable=False):
    top = tk.Toplevel(root)
    top.title(title)
    top.geometry(f"{w}x{h}")
    top.configure(bg=BG)
    top.resizable(resizable, resizable)
    top.grab_set()
    return top
```

The standard way every menu option opens its window: creates a new `Toplevel` (a real separate window, child of `root`), sets its title and pixel size, applies the dark background, sets whether it can be resized, and calls `grab_set()` to make it **modal** (input is locked to this window until it's closed). Nearly every `opt_*` function starts with `top = make_popup(...)`.

```
def style_treeview():
    s = ttk.Style()
    s.theme_use("clam")
    s.configure("D.Treeview",
                background=CARD, foreground=TEXT,
                fieldbackground=CARD, rowheight=30,
                font=("Segoe UI", 10))
    s.configure("D.Treeview.Heading",
                background=PANEL, foreground=CYAN,
                font=("Segoe UI", 10, "bold"))
    s.map("D.Treeview",
          background=[("selected", ACCENT)],
          foreground=[("selected", "white")])
```

Configures the dark-mode look for every `ttk.Treeview` (table) in the app. `ttk.Style()` is a **singleton** — calling it anywhere affects the whole app for the rest of its run — so `theme_use("clam")` here is what first switches the whole application off the native/Aqua theme onto "clam" (the theme that actually lets Tk repaint widget colors instead of using native OS chrome). `s.configure("D.Treeview", ...)` defines a *named style* (`"D.Treeview"`) with the dark background/text/row-height/font, `s.configure("D.Treeview.Heading", ...)` styles the column headers separately, and `s.map(..., background=[("selected", ACCENT)], ...)` sets what a *selected* row looks like (purple background, white text) — `.map()` is how `ttk` styles state-dependent appearance (selected/disabled/active) as opposed to the constant appearance set by `.configure()`.

```
def make_form_row(parent, label, row, show="", initial=""):
    tk.Label(parent, text=label, font=("Segoe UI", 11),
             fg=MUTED, bg=BG).grid(row=row, column=0, sticky="e", padx=(0, 12), pady=8)
    wrap = tk.Frame(parent, bg=CYAN, padx=1, pady=1)
    wrap.grid(row=row, column=1, sticky="ew", pady=8)
    e = tk.Entry(wrap, font=("Segoe UI", 12), bg=CARD, fg=TEXT,
                 insertbackground=CYAN, relief="flat", show=show, width=28)
    e.insert(0, initial)
    e.pack()
    return e
```

Builds one **grid row** of a form: a right-aligned label in column 0 and a cyan-bordered entry field in column 1 (same glow-border trick as `make_entry`, but grid-based since forms use `.grid()` for column alignment rather than `.pack()`). `e.insert(0, initial)` pre-fills the field — used by the Edit Student form to show the record's current values. Only the `Entry` widget is returned (not the wrapper), because form code only ever needs to call `.get()` / `.delete()` on it.

```
def build_treeview(parent, records, multi=False):
    style_treeview()
    cols   = ("student_id", "name", "major", "course_code", "course_title", "points", "gpa")
    heads  = ("Student ID", "Name", "Major", "Code", "Course Title", "Pts", "GPA")
    widths = (95, 130, 90, 68, 150, 48, 48)
```

```
    frame = tk.Frame(parent, bg=BG)
    frame.pack(fill="both", expand=True, padx=14, pady=(4, 0))

    tree = ttk.Treeview(frame, columns=cols, show="headings",
                        style="D.Treeview", height=10,
                        selectmode="extended" if multi else "browse")
    for col, head, w in zip(cols, heads, widths):
        tree.heading(col, text=head)
        tree.column(col, width=w, anchor="center")

    sb = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
    tree.configure(yscrollcommand=sb.set)
    sb.pack(side="right", fill="y")
    tree.pack(fill="both", expand=True)

    def refresh():
        tree.delete(*tree.get_children())
        for i, r in enumerate(records):
            tree.insert("", "end", iid=str(i),
                        values=(r["student_id"], r["name"], r["major"],
                                r["course_code"], r["course_title"],
                                r["points"], r["gpa"]))
    refresh()
    return tree, refresh
```

The shared table widget used by Edit Student and Delete Record.

- `cols` / `heads` / `widths` — three parallel tuples defining the 7 visible columns, their header text, and their pixel widths.
- `style_treeview()` applies the dark theme first.
- `ttk.Treeview(..., show="headings", ...)` creates the table with only column headings visible (no extra "tree" column); `selectmode` is `"extended"` (multi-select with Ctrl/Shift) when `multi=True` (Delete Record) or `"browse"` (single row) otherwise (Edit Student).
- The `for col, head, w in zip(...)` loop sets each column's header text and width/alignment in one pass.
- A vertical `ttk.Scrollbar` is wired to `tree.yview`, and the tree tells the scrollbar its position via `yscrollcommand=sb.set` — this is the standard two-way binding needed for scrollbars in Tkinter.
- `refresh()` is a nested function that clears every row (`tree.delete(*tree.get_children())`) and reinserts one row per record; `iid=str(i)` uses the record's **list index** as the row's internal id, which is exactly what lets the caller later do `idx = int(sel[0])` to map a selected row back to `records[idx]`.
- `refresh()` is called once immediately to populate the table, and the function returns both `tree` (for `.selection()` etc.) and `refresh` (so the caller can repopulate it after a save/delete without rebuilding the whole table).

---

11. Screen Helpers & Screens (Welcome → Register → Login → Menu)

```
def clear_screen():
    for w in root.winfo_children():
        w.destroy()
    root.unbind("<Key>")
```

Wipes the root window between "pages": `root.winfo_children()` lists every direct child widget of the root window, and each is `.destroy()`-ed (which also destroys their children automatically). `root.unbind("<Key>")` removes any keyboard shortcut that a previous screen registered (only `show_menu()` currently binds one, for the 0–9 shortcuts) so it can't fire on a screen where it no longer makes sense. Every `show_*` screen function calls this first.

```
def footer_close_btn(parent, cmd, label="×  Exit App"):
    bar = tk.Frame(parent, bg=PANEL)
    bar.pack(fill="x", side="bottom")
    tk.Frame(bar, bg=BORDER, height=1).pack(fill="x")
    make_button(bar, label, cmd, color="#374151", width=160, height=44, radius=22).pack(
        side="right", padx=24, pady=10)
```

A reusable "pinned to the bottom" bar with a single right-aligned button — a thin `BORDER`-colored divider line, then a dark gray rounded `make_button` running `cmd` when clicked. `side="bottom"` in `.pack()` is what pins it to the bottom of whatever `parent` it's given regardless of how much other content is packed above it (as long as that content isn't also using `side="bottom"`).

`show_welcome()`

```
def show_welcome():
    clear_screen()
    root.title("FEST Summer 2026")
    hdr = tk.Frame(root, bg=PANEL, height=200)
    hdr.pack(fill="x")
    hdr.pack_propagate(False)
    tk.Frame(hdr, bg=ACCENT, height=5).pack(fill="x")
    make_label(hdr, "FEST", 60, bold=True, color=ACCENT).pack(pady=(22, 0))
    make_label(hdr, "Summer 2026  ·  Vanderbilt University", 12, color=MUTED).pack()

    body = tk.Frame(root, bg=BG)
    body.pack(expand=True)
    make_label(body, "Register or Login to continue", 14, color=MUTED).pack(pady=(0, 36))
    make_button(body, "   Register   ", show_register, color=ACCENT,
                width=320, height=62, radius=31).pack(pady=10)
    make_button(body, "   Login   ", show_login, color=TEAL,
                width=320, height=62, radius=31).pack(pady=10)
```

```
    make_label(body, "Credentials stored in users.csv", 10, color=BORDER).pack(pady=(30, 0))

    footer_close_btn(root, root.destroy, label="×  Quit App")
```

The app's landing screen. `hdr.pack_propagate(False)` locks the header frame to its declared `height=200` regardless of what's packed inside it (without this call, a `Frame`'s size shrinks/grows to fit its children, which would make the header wobble as fonts render). The big "FEST" title and subtitle sit in that fixed-height header; below it, a centered body offers **Register** (purple) and **Login** (teal) buttons, whose `cmd` is literally the next screen function (`show_register` / `show_login`) — clicking either just calls that function directly, no event object needed since `make_button`'s `cmd` takes zero arguments. The footer's `root.destroy` quits the whole application.

`show_register()`

```
def show_register():
    clear_screen()
    ...
    def do_register():
        uid = r_id.get().strip()
        pw  = r_pw.get()
        pw2 = r_pw2.get()
        if not uid or not pw:
            messagebox.showerror("Error", "ID and password are required.")
            return
        if len(pw) < 4:
            messagebox.showerror("Error", "Password must be at least 4 characters.")
            return
        if pw != pw2:
            messagebox.showerror("Error", "Passwords do not match.")
            return
        if uid in load_users():
            messagebox.showerror("Error", f"ID '{uid}' is already taken.")
            return
        save_user(uid, hash_password(pw))
        messagebox.showinfo("Success", f"Account '{uid}' created!\nYou can now log in.")
        show_login()
```

Builds a title bar (`make_top_bar`) and three glow-bordered fields (`make_entry`, the third with `show="●"` to mask the password). The real logic lives in the nested `do_register()`, called when the **Register** button is clicked or Enter is pressed in the confirm-password field (`r_pw2.bind("<Return>", ...)`): it validates, in order, that both ID and password were entered, the password is at least 4 characters, the two password fields match, and the ID isn't already registered (`uid in load_users()`) — each failure shows an error and returns immediately (early-exit validation). If everything passes, it hashes the password, appends the new account via `save_user`, shows a success dialog, and sends the user straight to `show_login()`.

`show_login()`

```
def show_login():
    clear_screen()
    ...
    def do_login():
        global current_user
        uid = l_id.get().strip()
        pw  = l_pw.get()
        if not uid or not pw:
            messagebox.showerror("Error", "Enter your ID and password.")
            return
        users = load_users()
        if uid not in users or users[uid] != hash_password(pw):
            messagebox.showerror("Login Failed", "Invalid ID or password.")
            return
        current_user = uid
        show_menu()
```

Same layout pattern as Register, minus the confirm field. `do_login()` requires both fields to be non-empty, then loads all accounts and checks `uid not in users or users[uid] != hash_password(pw)` — this **re-hashes the entered password on every login attempt** and compares hash-to-hash, since the stored value is never a plaintext password to compare against directly. On success it sets the global `current_user` (this is why `global current_user` is declared) and moves to `show_menu()`.

`show_menu()`

```
def show_menu():
    clear_screen()
    root.title(f"FEST — Menu  ({current_user})")

    hdr = tk.Frame(root, bg=PANEL)
    hdr.pack(fill="x")
    tk.Frame(hdr, bg=ACCENT, height=4).pack(fill="x")
    row = tk.Frame(hdr, bg=PANEL)
    row.pack(fill="x", padx=20, pady=12)
    make_label(row, f"Logged in as:  {current_user}", 13,
               bold=True, color=CYAN).pack(side="left")
    make_button(row, "Logout", do_logout, color="#374151",
                width=120, height=42, radius=21).pack(side="right")

    body = tk.Frame(root, bg=BG)
    body.pack(expand=True, fill="both", padx=40, pady=20)
    make_label(body, "Main Menu  —  click a button or press 0–9",
               12, color=MUTED).pack(pady=(0, 18))

    grid = tk.Frame(body, bg=BG)
    grid.pack()
    for i in range(10):
```

```
        r, c = divmod(i, 2)
        color = OPTION_COLORS[i]
        border = tk.Frame(grid, bg=color, padx=2, pady=2)
        border.grid(row=r, column=c, padx=14, pady=10)
        inner = tk.Frame(border, bg=CARD)
        inner.pack()
        make_button(
            inner,
            text=f"  [ {i} ]   {OPTION_LABELS[i]}  ",
            cmd=lambda n=i: handle_option(n),
            color=CARD, fg=color,
            width=310, height=65, radius=0,
        ).pack()

    root.bind("<Key>", lambda e: (
        handle_option(int(e.char)) if e.char in "0123456789" else None
    ))

    footer_close_btn(root, root.destroy, label="×  Quit App")
```

A header shows who's logged in and a **Logout** button; the body draws a **2-column, 5-row grid of 10 option buttons** (`divmod(i, 2)` turns the flat index `i` (0–9) into a `(row, col)` pair — `0→(0,0)`, `1→(0,1)`, `2→(1,0)`, …, `9→(4,1)`). Each button is the "colored border → dark inner card → canvas button" pattern named in §4: a `color`-filled `Frame` acts as a 2px border, a `CARD`-colored `Frame` sits inside it, and the actual clickable `make_button` (flat, `radius=0`, so it fills the card edge-to-edge) sits inside that, with its **text colored** `color` and its **background** `CARD` — this is what gives each button its colored-outline-and-text-on-dark look instead of a solid color fill. `cmd=lambda n=i: handle_option(n)` captures `i` by value (`n=i` default argument) — without it every button's lambda would close over the same loop variable `i` and all ten buttons would end up calling `handle_option(9)` after the loop finished. `root.bind("<Key>", ...)` adds the keyboard shortcut: any digit `0`–`9` typed while the menu is focused calls `handle_option` with that digit — the digit check (`e.char in "0123456789"`) guards against non-digit keys throwing in `int(e.char)`.

```
def handle_option(n):
    dispatch = {
        0: opt_add_student,
        1: opt_edit_student,
        2: opt_delete_student,
        3: opt_gpa_formula,
        4: opt_swarm_plot,
        5: opt_countplot,
        6: opt_piechart,
        7: opt_student_profile,
        8: opt_student_id_card,
        9: opt_queries,
    }
    dispatch.get(n, lambda: None)()
```

The single dispatch point between "a button/key said option `n`" and "run the function for option `n`": a dict maps each integer 0–9 to its `opt_*` function object (not called yet — no parentheses in the dict), and `dispatch.get(n, lambda: None)()` looks up the function for `n` (or a harmless no-op `lambda: None` if `n` is somehow outside 0–9) and **then** calls it. Because Python resolves names in a dict literal at the time the dict is built, and `handle_option` is only ever *called* after the whole module (including every `opt_*` def below it) has finished loading, it's safe for this dispatch table to reference functions that are defined later in the file.

```
def do_logout():
    global current_user
    current_user = None
    show_welcome()
```

Clears the logged-in user and returns to the welcome screen — the exact inverse of what `do_login()` does.

---

12. Option 0 — Add Student

```
def opt_add_student():
    top = make_popup("Add New Student", w=560, h=560)
    ...
    field_defs = [
        ("Student ID",     ""), ("Full Name",      ""), ("Major",          ""),
        ("Course Code",    ""), ("Course Title",   ""), ("Points (0–100)", ""),
    ]
    entries = [make_form_row(form, lbl, i, initial=init)
               for i, (lbl, init) in enumerate(field_defs)]
    sid_e, name_e, major_e, code_e, title_e, pts_e = entries
```

Opens a popup and builds 6 form rows in one list comprehension using `make_form_row` (§10), then unpacks the returned `Entry` widgets into named variables for readability.

```
    gpa_var = tk.StringVar(value="—")
    ...
    def update_gpa(*_):
        try:
            letter, gpa = points_to_grade(pts_e.get())
            gpa_var.set(f"{letter}  /  {gpa:.1f}")
        except (ValueError, TypeError):
            gpa_var.set("—")
    pts_e.bind("<KeyRelease>", update_gpa)
```

A live grade preview: `gpa_var` is a `StringVar` bound to a label, and `update_gpa` recalculates it via `points_to_grade` every time a key is released inside the Points field (`<KeyRelease>`). If the field currently holds something non-numeric (e.g. empty, or mid-edit like `"9"` before `"90"`), `points_to_grade`'s internal `float(pts)` raises `ValueError`, which is caught here so the preview just shows `"—"` instead of crashing while

the user is still typing.

```
    def do_add():
        text_vals = [e.get().strip() for e in entries[:-1]]
        if not all(text_vals):
            messagebox.showerror("Error", "All fields are required.", parent=top)
            return
        try:
            pts = float(pts_e.get())
            assert 0 <= pts <= 100
        except (ValueError, AssertionError):
            messagebox.showerror("Error", "Points must be a number 0–100.", parent=top)
            return
        letter, gpa = points_to_grade(pts)
        append_student({
            "student_id":   text_vals[0], "name":  text_vals[1],
            "major":        text_vals[2], "course_code":  text_vals[3],
            "course_title": text_vals[4], "points": f"{pts:.1f}",
            "gpa":          f"{gpa:.1f}",
        })
        messagebox.showinfo("Saved", f"Student '{text_vals[1]}' added.\nGrade: {letter}  |
GPA: {gpa:.1f}", parent=top)
        for e in entries:
            e.delete(0, "end")
        gpa_var.set("—")
```

The save handler. `entries[:-1]` grabs every entry **except** the Points field (all text fields), and `all(text_vals)` fails validation if any of them is an empty string. Points is validated separately with `assert 0 <= pts <= 100` inside a `try` — both a bad number (`ValueError`) and an out-of-range number (`AssertionError`) are caught by the same `except` clause and reported as one message. On success, `points_to_grade` computes the final letter/GPA, `append_student` writes the new row, a confirmation dialog shows the computed grade, and every field is cleared (`e.delete(0, "end")`) so the form is ready for the next entry without closing the popup — this makes rapid batch entry (adding many rows for one student, or several students in a row) fast.

---

13. Option 1 — Edit Student

```
def opt_edit_student():
    top = make_popup("Edit Student Record", w=820, h=600)
    ...
    records = load_students()
    tree, refresh_tree = build_treeview(top, records)

    def open_edit_form():
        sel = tree.selection()
        if not sel:
```

```
            messagebox.showwarning("Select", "Select a row to edit.", parent=top)
            return
        idx = int(sel[0])
        rec = records[idx]
        win = tk.Toplevel(top)
        ...
```

Loads the whole roster once into `records` and displays it via `build_treeview` (§10). `open_edit_form()` reads the tree's current selection; recall from §10 that each row's `iid` was set to `str(list_index)`, so `int(sel[0])` recovers exactly which element of `records` was clicked, without needing to search by value. A second, nested `Toplevel` (`win`) — a popup on top of the popup — opens with a form pre-filled from `rec` via `make_form_row(..., initial=rec[key])`.

```
        def do_save():
            text_vals = {k: entries[k].get().strip()
                         for k in entries if k != "points"}
            if not all(text_vals.values()):
                messagebox.showerror("Error", "All fields are required.", parent=win)
                return
            try:
                pts = float(entries["points"].get())
                assert 0 <= pts <= 100
            except (ValueError, AssertionError):
                messagebox.showerror("Error", "Points must be 0–100.", parent=win)
                return
            letter, gpa = points_to_grade(pts)
            records[idx] = {
                "student_id": text_vals["student_id"], "name": text_vals["name"],
                "major": text_vals["major"], "course_code": text_vals["course_code"],
                "course_title": text_vals["course_title"],
                "points": f"{pts:.1f}", "gpa": f"{gpa:.1f}",
            }
            save_students(records)
            refresh_tree()
            messagebox.showinfo("Saved", "Record updated.", parent=win)
            win.destroy()
```

Same two-stage validation as Add Student (text fields non-empty, points a valid 0–100 number), then `records[idx] = {...}` **replaces the in-memory record at that exact index** with the edited values, `save_students(records)` rewrites the whole CSV from the updated list (§9), `refresh_tree()` redraws the table behind the edit popup so the change is visible immediately, and the edit popup closes itself.
The outer popup's own buttons — `"Edit Selected"` (calls `open_edit_form`) and `"Close"` (`top.destroy`) — are built the normal `make_button(...).pack(...)` way at the very end of the function.

---

14. Option 2 — Delete Student

```
def opt_delete_student():
    top = make_popup("Delete Student Records", w=820, h=560)
    ...
    records = load_students()
    tree, refresh_tree = build_treeview(top, records, multi=True)

    def do_delete():
        sel = tree.selection()
        if not sel:
            messagebox.showwarning("Select", "Select at least one record.", parent=top)
            return
        indices = {int(item) for item in sel}
        if not messagebox.askyesno(
            "Confirm Delete",
            f"Permanently delete {len(indices)} record(s)?\nThis cannot be undone.",
            parent=top,
        ):
            return
        new_recs = [r for i, r in enumerate(records) if i not in indices]
        records.clear()
        records.extend(new_recs)
        save_students(records)
        refresh_tree()
        messagebox.showinfo("Deleted", f"{len(indices)} record(s) removed.", parent=top)
```

`build_treeview(..., multi=True)` turns on `"extended"` selection mode so Ctrl/Shift-click can pick several rows. `indices = {int(item) for item in sel}` converts every selected row's `iid` back into a set of integer list-indices (a **set**, so membership testing with `in` is O(1) and duplicates from the selection API can't matter). `messagebox.askyesno(...)` blocks with a confirmation dialog — if the user clicks "No", the function returns and nothing is deleted. The actual removal, `[r for i, r in enumerate(records) if i not in indices]`, keeps every record **except** the ones whose index is in the selected set; `records.clear()` + `records.extend(new_recs)` mutates the existing `records` list object in place (rather than rebinding the name to a new list), which matters here only for consistency with the rest of the function — either approach would work since `records` isn't captured elsewhere by reference. `save_students` persists the shrunk list, `refresh_tree()` updates the visible table, and a summary dialog reports how many rows were removed.

---

15. Option 3 — GPA Formula (editable grading scale)

```
def opt_gpa_formula():
    top = make_popup("GPA Grading Scale", w=560, h=720)
    ...
    tree = ttk.Treeview(tree_frame, columns=("range", "letter", "gpa"),
                        show="headings", style="D.Treeview", height=13)
    ...
    def refresh_tree():
        tree.delete(*tree.get_children())
        for i, (lo, hi, letter, gpa) in enumerate(GPA_SCALE):
            tree.insert("", "end", iid=str(i),
                        values=(f"{lo} – {hi}", letter, f"{gpa:.1f}"))
    refresh_tree()
```

Unlike the student tables, this `Treeview` is built by hand (not via `build_treeview`, since its 3 columns are completely different) but reuses the same "row `iid` = list index" trick: `enumerate(GPA_SCALE)` gives each of the 13 grading bands an `iid` equal to its position in the list, so a selected row can be traced straight back to `GPA_SCALE[idx]`.

```
    selected_idx = [None]

    def on_select(event):
        sel = tree.selection()
        if not sel:
            return
        idx = int(sel[0])
        selected_idx[0] = idx
        lo, hi, letter, gpa = GPA_SCALE[idx]
        letter_var.set(letter)
        gpa_lbl_var.set(f"{gpa:.1f}")
        min_e.delete(0, "end"); min_e.insert(0, str(lo))
        max_e.delete(0, "end"); max_e.insert(0, str(hi))

    tree.bind("<<TreeviewSelect>>", on_select)
```

`selected_idx = [None]` is a one-element list used as a **mutable box** so the nested functions below (`on_select`, `save_row`) can both read and write "which row is currently selected" — a plain `selected_idx = None` local variable couldn't be reassigned from a nested function without an explicit `nonlocal` declaration, and using a list sidesteps that. `on_select` fires on the `<<TreeviewSelect>>` virtual event (Tkinter's built-in "selection changed" signal for `Treeview`) and copies the selected band's min/max/letter/GPA into the edit panel below the table so they can be changed. Note that **letter and GPA value are shown but not editable** here — only the point range (`min_e`, `max_e`) can be changed per row.

```
    def save_row():
        idx = selected_idx[0]
        if idx is None:
            messagebox.showwarning("Select", "Select a row first.", parent=top)
```

```
            return
        try:
            lo = int(min_e.get().strip())
            hi = int(max_e.get().strip())
            assert 0 <= lo <= 100 and 0 <= hi <= 100 and lo <= hi
        except (ValueError, AssertionError):
            messagebox.showerror("Invalid Input", "Min and Max must be whole numbers 0–100,
with Min ≤ Max.", parent=top)
            return
        _, _, letter, gpa = GPA_SCALE[idx]
        GPA_SCALE[idx] = (lo, hi, letter, gpa)
        save_gpa_scale()
        refresh_tree()
        tree.selection_set(str(idx))
        messagebox.showinfo("Saved", f"Range for {letter} updated to {lo} – {hi}.",
parent=top)
```

Validates that both bounds are whole numbers in `[0, 100]` with `lo <= hi`, then rebuilds the tuple at `GPA_SCALE[idx]` — keeping the existing `letter` / `gpa` (unpacked with `_` placeholders for the parts that don't change) but replacing `lo` / `hi` — persists it (`save_gpa_scale`), redraws the table, and re-selects the same row (`tree.selection_set(str(idx))`) so the user doesn't lose their place after the refresh wipes and rebuilds every row.

```
    def reset_defaults():
        if not messagebox.askyesno("Reset", "Reset all ranges back to defaults?",
parent=top):
            return
        GPA_SCALE.clear()
        GPA_SCALE.extend(DEFAULT_GPA_SCALE)
        save_gpa_scale()
        refresh_tree()
        selected_idx[0] = None
        letter_var.set("–"); gpa_lbl_var.set("–")
        min_e.delete(0, "end"); max_e.delete(0, "end")
        messagebox.showinfo("Reset", "Grading scale reset to defaults.", parent=top)
```

After confirmation, `GPA_SCALE.clear()` + `.extend(DEFAULT_GPA_SCALE)` restores the module-level list to the factory 13-band scale **in place** (so every other function that already imported/holds a reference to `GPA_SCALE` sees the change — important since `points_to_grade` reads this exact global), then clears the edit panel back to its empty state since nothing is selected anymore.

This screen directly controls the grading logic used everywhere else (`points_to_grade`) — so, for example, widening the `A` band here immediately changes what letter grade Option 0's live preview, Option 1's edit form, and Option 9's "A+" query all compute, the next time they run.

---

16. Option 4 — Swarm Plot

```
def opt_swarm_plot():
    records = load_students()
    if not records:
        messagebox.showwarning("No Data", ...); return
    try:
        pts = [float(r["points"]) for r in records]
    except ValueError:
        messagebox.showerror("Data Error", "Some records have invalid points values.")
        return
```

Loads the roster and bails out with a friendly warning if it's empty. Converting every `points` value to `float` up front, inside a `try`, means a single malformed row (e.g. someone hand-edited the CSV and left text in a numeric column) is caught **before** any chart drawing starts, rather than crashing partway through a Matplotlib call.

```
    codes = [r["course_code"] for r in records]
    cats  = [grade_category(p) for p in pts]

    fig, ax = plt.subplots(figsize=(9.2, 5.4))
    fig.patch.set_facecolor("#0d0d1a")
    ax.set_facecolor("#1a1a35")
    for spine in ax.spines.values():
        spine.set_color("#2a2a4a")
    ax.tick_params(colors="#94a3b8")
    ax.set_title(...); ax.set_xlabel(...); ax.set_ylabel(...)
    ax.set_ylim(-5, 108)
```

Builds one Matplotlib `Figure` / `Axes` pair and re-skins it to match the app's dark theme: figure background, axes background, the 4 border "spines," tick label color, and title/axis-label colors are all set explicitly (Matplotlib's defaults are white-background/black-text, which would clash badly with the rest of the UI). `ax.set_ylim(-5, 108)` gives a little headroom above 100 and below 0 so points sitting exactly on the axis limits aren't clipped.

```
    if HAS_SEABORN:
        df = pd.DataFrame({"Points": pts, "Course": codes, "Grade": cats})
        sns.swarmplot(data=df, x="Course", y="Points", hue="Grade",
                      palette=GRADE_PALETTE, ax=ax, size=9,
                      linewidth=0.5, edgecolor="#0d0d1a",
                      hue_order=list(GRADE_PALETTE.keys()))
        leg = ax.get_legend()
        if leg:
            leg.get_frame().set_facecolor("#16213e")
            ...
```

When Seaborn is available, the three parallel lists are packed into a `pandas.DataFrame` (Seaborn's `data=` API expects a DataFrame, not raw lists) and handed to `sns.swarmplot`, which does the actual hard part: spreading out points that would otherwise overlap at the same score, grouped by `Course` on the x-axis and colored by `Grade` using the fixed `GRADE_PALETTE`. The `if leg:` block then re-themes the auto-generated legend box (background, border, title, and label colors) to match the dark UI, since Seaborn/Matplotlib legends default to a white box.

```
    else:
        unique = sorted(set(codes))
        x_map  = {c: i for i, c in enumerate(unique)}
        rng    = np.random.default_rng(42)
        xs     = [x_map[c] + rng.uniform(-0.25, 0.25) for c in codes]
        colors = [GRADE_PALETTE[g] for g in cats]
        ax.scatter(xs, pts, c=colors, s=90, alpha=0.9,
                   edgecolors="#0d0d1a", linewidths=0.5)
        ax.set_xticks(range(len(unique))); ax.set_xticklabels(unique, color="#94a3b8")
        from matplotlib.patches import Patch
        handles = [Patch(facecolor=c, label=l) for l, c in GRADE_PALETTE.items()]
        leg = ax.legend(handles=handles, facecolor="#16213e", edgecolor="#2a2a4a")
        for t in leg.get_texts():
            t.set_color("#f1f5f9")
```

The fallback used when Seaborn isn't installed, hand-rolling a similar effect: `x_map` assigns each unique course code an integer x-position, `np.random.default_rng(42)` creates a **seeded** random generator (fixed seed `42`, so the horizontal jitter is reproducible between runs rather than reshuffling every time the chart is redrawn) used to nudge each point's x-coordinate by up to ±0.25 so points at the same score/course don't stack exactly on top of each other — a manual approximation of what `swarmplot` does automatically. Colors come straight from `GRADE_PALETTE` keyed by each point's `grade_category`. Since a plain `ax.scatter` doesn't auto-generate a legend, one is built by hand from `matplotlib.patches.Patch` objects (one colored square per grade band) and then re-themed the same way as the Seaborn branch.

```
    for threshold, color in [(90, "#10b981"), (80, "#3b82f6"),
                             (70, "#f59e0b"), (60, "#f97316")]:
        ax.axhline(threshold, color=color, linewidth=0.7, linestyle="--", alpha=0.45)
    plt.tight_layout(pad=1.5)
```

Draws four faint dashed horizontal reference lines at the grade-band cutoffs (90/80/70/60), colored to match each band, so it's visually obvious which side of a cutoff any given point falls on regardless of which rendering branch drew it.

```
    toolbar_frame = tk.Frame(top, bg="#16213e")
    toolbar_frame.pack(fill="x", side="bottom")
    def on_close():
        plt.close(fig)
        top.destroy()
    make_button(toolbar_frame, "Close", on_close, ...).pack(side="right", padx=12, pady=6)

    canvas = FigureCanvasTkAgg(fig, master=top)
    NavigationToolbar2Tk(canvas, toolbar_frame).update()
    canvas.get_tk_widget().pack(fill="both", expand=True)
    canvas.draw()
    top.protocol("WM_DELETE_WINDOW", on_close)
```

Embeds the finished Matplotlib figure directly inside the Tkinter popup instead of opening a separate Matplotlib window: `FigureCanvasTkAgg(fig, master=top)` renders the figure onto a Tkinter-compatible canvas widget, and `NavigationToolbar2Tk` adds Matplotlib's standard pan/zoom/save toolbar underneath it. `on_close()` is defined once and used in **two** places — the custom "Close" button and `top.protocol("WM_DELETE_WINDOW", on_close)` (which intercepts the OS window's own close button/red-X) — so that however the user closes the window, `plt.close(fig)` always runs first to free the figure's memory before the Tkinter window is destroyed (without it, repeatedly opening/closing this chart would leak `Figure` objects).

---

17. Option 5 — Major Count Plot

```
def opt_countplot():
    records = load_students()
    if not records:
        messagebox.showwarning(...); return
    ...
    major_counts = {}
    for r in records:
        major = r["major"].strip()
        major_counts[major] = major_counts.get(major, 0) + 1
    majors = list(major_counts.keys())
    counts = [major_counts[m] for m in majors]
```

Same empty-roster guard as the swarm plot, then a manual tally: `major_counts.get(major, 0) + 1` is the standard "increment-or-initialize" pattern for building a frequency count with a plain dict (no `collections.Counter` needed for something this small). `.strip()` guards against a major value with stray leading/trailing whitespace being counted as a separate "major" from its trimmed version. Note this counts **enrollment rows**, not unique students — a student with 4 course rows contributes 4 to their major's count.

```
    if HAS_SEABORN:
        df = pd.DataFrame({"Major": majors, "Count": counts})
        df = df.sort_values("Count", ascending=False)
        sns.barplot(data=df, x="Major", y="Count", palette="viridis",
                    ax=ax, edgecolor="#0d0d1a", linewidth=0.8)
        for bar, count in zip(ax.patches, df["Count"]):
            ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.05,
                    str(count), ha="center", va="bottom", ...)
```

Packs the tally into a small DataFrame, sorts it descending by count (so the tallest bar is always leftmost), and lets Seaborn draw the bars with its built-in `"viridis"` color ramp. The `for bar, count in zip(...)` loop then manually stamps the numeric count just above each bar — Seaborn doesn't add data labels automatically, so this loop pairs each drawn bar patch with its count and places a centered text label just above its top edge.

```
    else:
        bar_colors = ["#7c3aed", "#0d7377", "#b45309", "#1d4ed8",
                      "#be185d", "#065f46", "#92400e", "#1e3a5f"]
        bars = ax.bar(majors, counts,
                      color=[bar_colors[i % len(bar_colors)] for i in range(len(majors))],
                      edgecolor="#0d0d1a", linewidth=0.8)
        for bar, count in zip(bars, counts):
            ax.text(...)
        ax.set_xticks(range(len(majors))); ax.set_xticklabels(majors, color="#94a3b8")
```

The no-Seaborn fallback: plain `ax.bar`, cycling through a fixed 8-color palette with `i % len(bar_colors)` so it never runs out of colors even with more than 8 majors, and the same manual count-label loop as above (duplicated here rather than shared, since the two branches use different Matplotlib objects — `ax.patches` from Seaborn vs. the `bars` container `ax.bar` returns directly).

```
    ax.yaxis.set_major_locator(plt.MaxNLocator(integer=True))
    ax.set_ylim(0, max(counts) + 1.5)
    plt.xticks(rotation=20, ha="right")
    plt.tight_layout(pad=1.5)
```

`MaxNLocator(integer=True)` forces the y-axis ticks to whole numbers only (since "number of students" can't be fractional, e.g. no `2.5` tick); `set_ylim(0, max(counts) + 1.5)` leaves headroom above the tallest bar so its count label isn't clipped at the top edge; rotating the x-tick labels 20° with right alignment keeps longer major names (like "Mechanical Engineering") from overlapping each other. The rest of the function (toolbar, embed-in-Tkinter, `on_close`) is identical in structure to Option 4 — see §16 for that pattern.

---

18. Option 6 — Grade Pie Chart

```
def opt_piechart():
    records = load_students()
```

```
    if not records:
        messagebox.showwarning(...); return

    grade_counts = {}
    for r in records:
        try:
            letter, _ = points_to_grade(float(r["points"]))
            key = letter[0]          # collapse A+/A/A- → "A", etc.
        except (ValueError, TypeError):
            key = "F"
        grade_counts[key] = grade_counts.get(key, 0) + 1

    ordered_keys = [k for k in ["A", "B", "C", "D", "F"] if k in grade_counts]
    labels  = [f"Grade {k}" for k in ordered_keys]
    sizes   = [grade_counts[k] for k in ordered_keys]
    colors  = {"A": "#10b981", "B": "#3b82f6", "C": "#f59e0b", "D": "#f97316", "F":
"#ef4444"}
    pie_colors  = [colors[k] for k in ordered_keys]
```

Tallies each row into one of 5 coarse letter buckets: `points_to_grade` gives the precise letter (e.g. `"A-"`), and `letter[0]` collapses it down to just its first character so `A+` / `A` / `A-` all count toward the same `"A"` slice. A row with an unparseable `points` value is defensively counted as an `"F"` rather than skipped or crashing. `ordered_keys` filters the fixed `["A","B","C","D","F"]` sequence down to only the grades actually present, which is what keeps the slice order (and legend order) always A→F regardless of dict insertion order, while never drawing an empty 0% slice for a grade nobody got.

```
    # Only pull out the single largest slice, keep the rest flush.
    explode_idx = sizes.index(max(sizes))
    explode     = [0.08 if i == explode_idx else 0 for i in range(len(sizes))]
```

`sizes.index(max(sizes))` finds the position of the largest slice (the most common grade); the list comprehension then builds an `explode` array — Matplotlib's per-slice "pull outward" distance — that is `0.08` for that one slice and `0` for every other, so exactly one wedge is ever pulled out of the pie, regardless of how many grade bands exist.

```
    def _darken(hex_color, factor=0.55):
        hex_color = hex_color.lstrip("#")
        r, g, b = (int(hex_color[i:i+2], 16) for i in (0, 2, 4))
        r, g, b = int(r * factor), int(g * factor), int(b * factor)
        return f"#{r:02x}{g:02x}{b:02x}"
```

A small hex-color helper: strips the leading `#`, splits the 6-character hex string into its `r` / `g` / `b` byte pairs (`hex_color[0:2]`, `[2:4]`, `[4:6]`) and parses each as base-16, multiplies every channel by `factor` (`0.55` = 45% darker) to darken the color, then reassembles it back into a `#rrggbb` string with `f"#{r:02x}{g:02x}{b:02x}"` (`02x` = 2-digit lowercase hex, zero-padded).

```
    shade_colors = [_darken(c) for c in pie_colors]
    DEPTH_LAYERS = 14
    DEPTH_STEP   = 0.012
    for layer in range(DEPTH_LAYERS, 0, -1):
        ax.pie(sizes, explode=explode, colors=shade_colors, startangle=140,
               center=(0, -DEPTH_STEP * layer), radius=1,
               wedgeprops={"edgecolor": "none"})
```

This is the fake-3D trick: Matplotlib has no native 3D pie chart, so the "depth" is faked by drawing 14 extra copies of the *same* pie in darkened colors, each one nudged a little further down (`center=(0, -DEPTH_STEP * layer)`), from the **furthest-down copy first up to the least-offset copy**. Since each later `ax.pie(...)` call draws on top of the previous ones, layers closer to the real position end up covering more of the layers further below them, leaving only a thin sliver of each visible — the same technique used to fake extruded "3D bar" or "3D pie" charts in tools like Excel, just implemented by hand here.

```
    wedges, texts, autotexts = ax.pie(
        sizes, labels=None,
        autopct=lambda pct: f"{pct:.1f}%\n({round(pct * sum(sizes) / 100)})",
        colors=pie_colors, explode=explode, startangle=140, shadow=True,
        wedgeprops={"edgecolor": "#0d0d1a", "linewidth": 2},
        pctdistance=0.72, textprops={"fontsize": 11},
    )
    ax.set_ylim(-1.2 - DEPTH_STEP * DEPTH_LAYERS, 1.2)
```

The **real** pie, drawn last (so it sits on top of all 14 shadow layers) in the actual bright `pie_colors`. `autopct=lambda pct: ...` is Matplotlib's hook for generating each slice's percentage label — here it's overridden to show both the percentage **and** the raw student count on two lines (`pct * sum(sizes) / 100` converts the percentage back into an absolute count). `shadow=True` layers Matplotlib's own built-in drop shadow on top of the hand-drawn extrusion for extra depth. `ax.set_ylim(...)` is expanded downward by `DEPTH_STEP * DEPTH_LAYERS` so the lowest shadow layer's peeking sliver isn't clipped by the default axis limits.

```
    legend_labels = [f"Grade {k}  ({grade_counts[k]} student{'s' if grade_counts[k] != 1 else
''})"
                     for k in ordered_keys]
    leg = ax.legend(wedges, legend_labels, loc="center left",
                    bbox_to_anchor=(1.02, 0.5), frameon=True, framealpha=0.9,
                    facecolor="#16213e", edgecolor="#2a2a4a", fontsize=11)
```

Builds a custom legend that pairs each `wedge` patch (returned from the real `ax.pie(...)` call) with a label like `"Grade A  (42 students)"` — handling the singular/plural `'s'` inline. `bbox_to_anchor=(1.02, 0.5)` positions the legend just outside the right edge of the axes (x slightly past 1.0) so it never overlaps the pie itself.

---

19. Option 7 — Student Profile Viewer

```
def opt_student_profile():
    records = load_students()
    if not records:
        messagebox.showwarning(...); return

    student_dict = {}
    for r in records:
        sid = r["student_id"]
        if sid not in student_dict:
            student_dict[sid] = []
        student_dict[sid].append(r)

    student_ids  = list(student_dict.keys())
    total        = len(student_ids)
    current_idx  = [0]
```

The roster is flat (one row per enrollment), so the first thing this screen does is **re-group it by student**: `student_dict` becomes `{student_id: [row, row, ...]}`. `student_ids` is that dict's keys as a list (preserving first-seen order, since Python dicts are insertion-ordered), and `current_idx = [0]` is the same "mutable one-element list as a box" trick used in Option 3 — it lets nested functions both read and reassign "which student index is currently shown" without needing `nonlocal`.

```
    def do_search():
        sid = srch_entry.get().strip()
        if not sid:
            return
        if sid in student_dict:
            show_student(student_ids.index(sid))
        else:
            messagebox.showwarning("Not Found", f"Student ID  '{sid}'  not found.",
parent=top)
```

Looks the typed ID up directly in `student_dict` (an O(1) dict lookup); if found, `student_ids.index(sid)` converts the ID back into its position in the `student_ids` list so `show_student` (which navigates by index) can jump straight to it.

```
    prev_btn = make_button(inner_nav, "◀  Previous", None, ...)
    ...
    next_btn = make_button(inner_nav, "Next  ▶", None, ...)
    ...
    def show_student(idx):
        current_idx[0] = idx
        sid  = student_ids[idx]
        recs = student_dict[sid]
        first = recs[0]
```

```
        id_var.set(f"  {first['student_id']}")
        name_var.set(f"  {first['name']}")
        major_var.set(f"  {first['major']}")

        tree.delete(*tree.get_children())
        for r in recs:
            try:
                letter, _ = points_to_grade(float(r["points"]))
            except (ValueError, TypeError):
                letter = "—"
            tree.insert("", "end", values=(r["course_code"], r["course_title"], r["points"],
letter, r["gpa"]))

        counter_var.set(f"{idx + 1}  /  {total}")

        prev_btn.bind("<Button-1>",
                      lambda e: show_student(current_idx[0] - 1) if current_idx[0] > 0 else
None)
        next_btn.bind("<Button-1>",
                      lambda e: show_student(current_idx[0] + 1) if current_idx[0] < total -
1 else None)

        _dim_nav()
```

Note that `prev_btn` / `next_btn` are created with `cmd=None` — they get **no** click behavior at creation time. Instead, every time `show_student(idx)` runs, it re-binds each button's `<Button-1>` click event with a fresh lambda that already knows the *current* boundary condition (`current_idx[0] > 0` / `current_idx[0] < total - 1`). This re-binding approach exists because these buttons are plain `Canvas` objects from `make_button` (§10) which have no real "disabled" state the way a native widget would — so instead of disabling the button, the click handler itself refuses to navigate past the first/last student (returning `None` instead of calling `show_student`), and `_dim_nav()` (below) repaints the button to visually look disabled to match.

`first = recs[0]` — identity/major info (name, major, ID) is only shown from the *first* enrollment row, since those fields are duplicated identically across every row for the same student. The course table underneath, however, iterates **all** of `recs`, since that's the whole point of the table — showing every course that student is enrolled in, with a freshly computed letter grade per row (falling back to `"—"` if a row's `points` can't be parsed).

```
    def _dim_nav():
        idx = current_idx[0]
        _set_btn_alpha(prev_btn, active=(idx > 0))
        _set_btn_alpha(next_btn, active=(idx < total - 1))

    def _set_btn_alpha(btn_canvas, active):
        color = OPTION_COLORS[7] if active else "#374151"
        btn_canvas.delete("all")
        ...
        btn_canvas.create_text(w // 2, h // 2, text=label,
                               fill="white" if active else MUTED, ...)
```

`_set_btn_alpha` is a hand-rolled "enabled vs. disabled" repaint for one of these canvas buttons: it clears the canvas and redraws the exact same rounded-rectangle shape as `make_button`'s internal `_draw()` (§10), but picks the fill color and text color based on the `active` flag — the accent color with white text when usable, or dark gray with muted text when at a boundary. This is necessary specifically because these buttons are hand-drawn on a `Canvas` rather than a real Tkinter widget, so there's no built-in `state="disabled"` to lean on (contrast with Option 8's Previous/Next buttons, which — after being switched to `ttk.Button` — *do* use real disabled styling; see §20).

```
    show_student(0)
```

Shows the first student in the roster the moment the popup opens, exactly like the pattern used at the end of every other browsable-list screen in the app.

---

20. Option 8 — Student ID Card (Vanderbilt style)

```
def opt_student_id_card():
    records = load_students()
    if not records:
        messagebox.showwarning(...); return

    student_dict = {}
    for r in records:
        sid = r["student_id"]
        if sid not in student_dict:
            student_dict[sid] = []
        student_dict[sid].append(r)

    student_ids = list(student_dict.keys())
    total       = len(student_ids)
    current     = [0]

    VU_GOLD  = "#CFAE70"
    VU_BLACK = "#121212"
```

```
    CARD_W   = 520
    CARD_H   = 295
```

Same "re-group the flat roster by student" step as Option 7, plus four constants specific to the ID card's visual design: Vanderbilt's gold/black color pair, and the fixed pixel dimensions of the card graphic itself.

`make_avatar` — a procedurally generated cartoon face

```
def make_avatar(parent, name, size=110):
    import math
    seed = int(hashlib.md5(name.encode()).hexdigest(), 16)
    SKIN  = [...]; HAIR  = [...]; SHIRT = [...]; IRIS  = [...]
    skin    = SKIN [(seed)       % len(SKIN)]
    hair    = HAIR [(seed >>  8) % len(HAIR)]
    shirt   = SHIRT[(seed >> 16) % len(SHIRT)]
    iris    = IRIS [(seed >> 20) % len(IRIS)]
    hstyle = (seed >> 12) % 3
    glasses  = ((seed >> 24) % 4) == 0
    freckles = ((seed >> 28) % 5) == 0
```

Rather than storing or generating a real photo, every student gets a **deterministic cartoon avatar** derived purely from their name. `hashlib.md5(name.encode()).hexdigest()` hashes the student's name into a fixed-length hex string, and `int(..., 16)` turns that into one large integer — the `seed`. Every visual trait then picks an option from a small palette list by taking a different **bit-shifted slice** of that same seed and reducing it with `% len(list)`: `seed % len(SKIN)` uses the low bits for skin tone, `seed >> 8` shifts off the bottom 8 bits before reducing (so hair color is decided by a different part of the hash than skin tone), and so on up through `seed >> 28` for whether freckles appear. The net effect: the exact same name **always** produces the exact same avatar (useful — the same student looks the same every time their card is viewed), while different names are extremely likely to land on different combinations, since MD5 output is effectively random-looking across its bits.

```
    cv = tk.Canvas(parent, width=size, height=size, bg=VU_BLACK, highlightthickness=0)
    cx = size // 2
    cv.create_oval(2, 2, size-2, size-2, fill="#1a2233", outline=VU_GOLD, width=2)
    ...
    cv.create_rectangle(cx-nw, int(size*.57), cx+nw, by_, fill=skin, outline="")
    ...
    if hstyle == 0:
        cv.create_arc(...)          # short/cropped hair
    elif hstyle == 1:
        for ang in range(0, 181, 36):
            ...                      # curly hair (a ring of small circles)
    else:
        cv.create_oval(...)          # long/full hair
    cv.create_oval(hcx-hr, hcy-hr, hcx+hr, hcy+hr, fill=skin, outline="")
    ...
```

The rest of the function is a sequence of `Canvas` drawing primitives building the face layer by layer, back to front: a circular gold-rimmed frame, a colored shirt/shoulders shape, a neck rectangle, then a head — with the actual hairstyle chosen by `hstyle` (0/1/2, picked from the seed) branching into three different drawing routines (a simple arc for short hair, a ring of small circles for a curly look, or one big oval for long hair) before the head circle itself is drawn on top of the hair so it isn't hidden underneath it. Eyes, eyebrows, nose, and mouth are all drawn with a handful more `create_oval` / `create_arc` / `create_line` calls using proportional offsets (`hr * .22`, `hr * .36`, etc.) so the whole face scales cleanly if `size` is ever changed. `if freckles:` / `if glasses:` at the end conditionally layer on two more optional decorative details, again gated by bits of the same seed.

`make_barcode` — a fake but visually convincing barcode

```
def make_barcode(parent, seed_str, w=CARD_W, h=34):
    seed = int(hashlib.md5(seed_str.encode()).hexdigest(), 16) & 0xFFFFFFFF
    rng  = np.random.default_rng(seed)
    cv   = tk.Canvas(parent, width=w, height=h, bg=VU_GOLD, highlightthickness=0)
    x = 10
    while x < w - 10:
        bw = int(rng.choice([1, 1, 2, 2, 3]))
        gp = int(rng.choice([1, 2, 3]))
        cv.create_rectangle(x, 3, x+bw, h-8, fill=VU_BLACK, outline="")
        x += bw + gp
    cv.create_text(w//2, h-3, text=seed_str, anchor="s", fill=VU_BLACK, font=("Courier New",
7, "bold"))
    return cv
```

Same deterministic-hash idea as the avatar, but seeded from the **student ID** instead of the name, and this time feeding the hash into `np.random.default_rng(seed)` (a proper seeded NumPy random generator, masked to 32 bits with `& 0xFFFFFFFF` since the generator expects an integer seed in that range) rather than reading individual bits by hand. The `while x < w - 10:` loop walks left to right across the barcode's width, at each step picking a random bar width (weighted toward thin bars via the repeated `1, 1` in the choice list) and a random gap, drawing a black rectangle, and advancing `x` past it — repeating until the available width runs out. The student's ID number is printed underneath in a monospace font, same as a real barcode label.

`populate_card` — filling in the actual card

```
def populate_card(card_body, sid):
    for w in card_body.winfo_children():
        w.destroy()
    first = student_dict[sid][0]
    recs  = student_dict[sid]
    ...
    make_avatar(left, first["name"]).pack()
    ...
    tk.Label(right, text=first["student_id"], font=("Courier New", 15, "bold"),
...).pack(anchor="w")
    tk.Label(right, text=f"Enrolled courses: {len(recs)}", ...).pack(anchor="w", pady=(6, 0))
    make_barcode(card_body, first["student_id"]).pack(fill="x")
```

This is what actually gets called every time the displayed student changes. It first **destroys every existing child widget** of `card_body` (`for w in card_body.winfo_children(): w.destroy()`) — rather than creating a brand-new card frame for every student, the app reuses one fixed-size frame and completely re-populates it each time, which is why the card's outer size never jitters as different students (with different name lengths, course counts, etc.) are shown. It then rebuilds the gold header bar, the avatar + "STUDENT" label on the left, a vertical gold divider, and the name/major/university line/ID/course-count block on the right, finishing with a freshly generated barcode along the bottom.

```
    top = tk.Toplevel(root)
    ...
    top.geometry("780x600")
    ...
    card_outer = tk.Frame(top, bg=VU_GOLD)
    card_outer.pack(pady=24)
    card_body = tk.Frame(card_outer, bg=VU_BLACK, width=CARD_W, height=CARD_H)
    card_body.pack(padx=3, pady=3)
    card_body.pack_propagate(False)
```

The popup itself is fixed at `780x600` (enlarged from an original, cramped `660x530` after user feedback that the Close button was hard to see — see §21 note below). `card_outer` is a thin gold `Frame` acting as the card's outer border; `card_body` is the actual black card surface sized to the exact `CARD_W × CARD_H` "physical ID card" dimensions, with `pack_propagate(False)` locking that size regardless of how much or little content `populate_card` puts inside it.

```
    nav_style = ttk.Style()
    nav_style.theme_use("clam")
    nav_style.configure("Nav.TButton",
                        font=("Segoe UI", 12, "bold"),
                        background="#121212", foreground="white",
                        bordercolor="#121212", focuscolor="#121212",
                        padding=(22, 11), relief="flat")
    nav_style.map("Nav.TButton",
                  background=[("disabled", "#374151"), ("active", "#2a2a2a")],
```

```
                  foreground=[("disabled", "#8a8a8a"), ("active", "white")])

    prev_btn = ttk.Button(inner_nav, text="◀  Previous", style="Nav.TButton",
                          cursor="hand2", command=lambda: show_card(current[0] - 1))
    ...
    next_btn = ttk.Button(inner_nav, text="Next  ▶", style="Nav.TButton",
                          cursor="hand2", command=lambda: show_card(current[0] + 1))
```

Unlike Option 7's hand-painted Canvas buttons, the Previous/Next buttons here are **real** `ttk.Button` widgets — but styled through a custom `ttk.Style` named `"Nav.TButton"` rather than left at their default appearance. This exists because of a real, discovered bug: a plain `tk.Button` **ignores** its `bg`/`activebackground` options on macOS, since the Aqua theme renders buttons using native OS chrome and refuses to let Tk recolor them — so no matter what color was set, the button kept showing up with a native white/gray face. Switching the whole app's `ttk` theme to `"clam"` (which `style_treeview()`, §10, already does elsewhere, and which this function forces again defensively with `theme_use("clam")`) makes `ttk` paint its own button face in software instead of deferring to native rendering, so the configured colors — dark `#121212` normally, lighter `#2a2a2a` on hover/active, muted `#374151` when `disabled` — actually show up. `nav_style.map(...)` is what ties each color to a **state** (`"disabled"`, `"active"`) rather than a constant, which also means the boundary-dimming here can just be `prev_btn.config(state="disabled")` — a real widget state, unlike Option 7's manual redraw workaround.

```
    def show_card(i):
        current[0] = i
        populate_card(card_body, student_ids[i])
        counter_var.set(f"{i + 1}  /  {total}")
        prev_btn.config(state="normal" if i > 0       else "disabled")
        next_btn.config(state="normal" if i < total-1 else "disabled")

    show_card(0)   # first student shown by default
```

The screen's own navigation function: update the tracked index, repopulate the one reusable card frame for the new student, update the "`n / total`" counter label, and flip each button's real `state` between `"normal"`/`"disabled"` based on whether moving further in that direction is still valid. `show_card(0)` at the end displays the first student immediately when the popup opens — this exact line was, at one point, unreachable because an invalid `disabledbackground` option on an earlier version of these buttons raised an error during widget creation and silently aborted the rest of the function before this line ever ran; that bug is why the Previous/Next buttons were rebuilt as `ttk.Button`s in the first place.

---

21. Option 9 — Queries

```
def opt_queries():
    records = load_students()
    if not records:
        messagebox.showwarning(...); return

    student_dict = {}
    for r in records:
        student_dict.setdefault(r["student_id"], []).append(r)
```

Same student-grouping step as Options 7/8, written slightly more tersely with `dict.setdefault(key, []).append(r)` — a one-line idiom equivalent to "if the key isn't there yet, create an empty list for it, then append to whatever list is there."

```
    def grade_letter(r):
        try:
            letter, _ = points_to_grade(float(r["points"]))
            return letter
        except (ValueError, TypeError):
            return None

    def distinct_students(pred):
        return len({r["student_id"] for r in records if pred(r)})

    def avg_points(pred):
        vals = [float(r["points"]) for r in records if pred(r)]
        return sum(vals) / len(vals) if vals else None

    def avg_gpa(pred):
        vals = [float(r["gpa"]) for r in records if pred(r)]
        return sum(vals) / len(vals) if vals else None

    def plural(n, word="student"):
        return f"{n} {word}{'s' if n != 1 else ''}"
```

Four small, reusable query-building blocks, each taking a **predicate function** (`pred`, a function from one row-dict to `True`/`False`) so the same helper can answer many different questions just by passing a different filter:

- `distinct_students(pred)` — a **set comprehension** collects the `student_id` of every row matching `pred`, and `len(...)` counts the *unique* students (not enrollment rows) that satisfy it.
- `avg_points(pred)` / `avg_gpa(pred)` — collect the matching rows' numeric `points`/`gpa` values and average them, returning `None` (rather than raising a division-by-zero error) if nothing matched.
- `plural(n, word)` — a tiny grammar helper so answers read as "1 student" vs. "3 students" instead of always pluralizing.

```
q1 = distinct_students(lambda r: r["major"] == "Computer Science"
                                 and grade_letter(r) == "A+")

q2 = distinct_students(lambda r: r["major"] == "Mechanical Engineering"
                                 and "program" in r["course_title"].lower())

q3 = distinct_students(lambda r: "program" in r["course_title"].lower())
```

The three questions the project was explicitly asked to include: **Q1** counts unique Computer Science students who earned an A+ in *any* course; **Q2** counts unique Mechanical Engineering students enrolled in a course whose title contains "program" (case-insensitively, via `.lower()`, so it matches "Python Programming" regardless of capitalization) — i.e., a programming/CS course; **Q3** drops the major filter entirely and counts every student, from any major, in a programming-titled course.

```
v4 = avg_gpa(lambda r: r["major"] == "Computer Science")
q4 = f"{v4:.2f} GPA" if v4 is not None else "No data"

majors = sorted({r["major"] for r in records})
major_gpa = {m: avg_gpa(lambda r, m=m: r["major"] == m) for m in majors}
major_gpa = {m: v for m, v in major_gpa.items() if v is not None}
if major_gpa:
    top_major = max(major_gpa, key=major_gpa.get)
    q5 = f"{top_major}  ({major_gpa[top_major]:.2f} avg GPA)"
else:
    q5 = "No data"
```

Q4 is a single `avg_gpa` call filtered to Computer Science. **Q5** ("which major has the highest average GPA") first collects every distinct major, then builds `{major: avg_gpa_for_that_major}` with a dict comprehension — note `lambda r, m=m: ...` uses the same "capture-by-default-argument" trick from `show_menu()`'s button loop (§11), needed here because the lambda is built fresh on every iteration of the comprehension and would otherwise all end up referencing whatever `m` was on the *last* iteration. Majors with no valid GPA data are filtered out, and `max(major_gpa, key=major_gpa.get)` finds the dict key whose value is largest (i.e., the major with the best average GPA) without needing to sort the whole thing.

```
q6 = distinct_students(lambda r: r["course_title"] == "Circuit Analysis"
                                 and grade_letter(r) == "F")
q7 = len(student_dict)
v8 = avg_points(lambda r: r["course_title"] == "Calculus I")
q8 = f"{v8:.1f} / 100" if v8 is not None else "No data"
q9 = distinct_students(lambda r: r["course_title"] == "Introduction to Business"
                                 and float(r["points"]) >= 90)
```

Q6–Q9 reuse the same four helper functions against different course/threshold combinations: students who failed Circuit Analysis, the total number of unique students in the whole roster (`len(student_dict)` — no filtering needed, it's already grouped by student), the class average in Calculus I, and how many students scored 90+ in Introduction to Business.

```
    titles = sorted({r["course_title"] for r in records})
    title_pts = {t: avg_points(lambda r, t=t: r["course_title"] == t) for t in titles}
    title_pts = {t: v for t, v in title_pts.items() if v is not None}
    if title_pts:
        low_title = min(title_pts, key=title_pts.get)
        q10 = f"{low_title}  ({title_pts[low_title]:.1f} avg pts)"
    else:
        q10 = "No data"
```

Q10 mirrors Q5's pattern exactly, but for courses instead of majors, and `min(...)` instead of `max(...)` — finding the course whose average score is lowest.

```
    QUERIES = [
        ("How many students from Computer Science earned an A+?", plural(q1)),
        ...
        ("Which course has the lowest average score overall?", q10),
    ]
```

All ten `(question, computed_answer)` pairs are collected into one list, which is what the UI-building loop below iterates over.

```
    canvas = tk.Canvas(container, bg=BG, highlightthickness=0)
    sb = ttk.Scrollbar(container, orient="vertical", command=canvas.yview)
    scroll_frame = tk.Frame(canvas, bg=BG)
    scroll_frame.bind("<Configure>", lambda e:
canvas.configure(scrollregion=canvas.bbox("all")))
    canvas.create_window((0, 0), window=scroll_frame, anchor="nw")
    canvas.configure(yscrollcommand=sb.set)
    canvas.pack(side="left", fill="both", expand=True)
    sb.pack(side="right", fill="y")

    def _on_mousewheel(event):
        canvas.yview_scroll(int(-1 * (event.delta / 120)), "units")
    canvas.bind("<Enter>", lambda e: canvas.bind_all("<MouseWheel>", _on_mousewheel))
    canvas.bind("<Leave>", lambda e: canvas.unbind_all("<MouseWheel>"))
```

Tkinter has no built-in "scrollable frame" widget, so this is the standard workaround: put an ordinary `Frame` (`scroll_frame`) *inside* a `Canvas` (via `canvas.create_window`), and let the `Scrollbar` drive the canvas's viewport (`canvas.yview`) rather than the frame directly. `scroll_frame.bind("<Configure>", ...)` re-measures the frame's full size every time its contents change and updates the canvas's `scrollregion`

accordingly, which is what lets the scrollbar's range correctly reflect exactly 10 stacked question cards, however tall they end up being. The mouse-wheel handler is only active while the cursor is actually over this canvas (`bind_all` on `<Enter>`, `unbind_all` on `<Leave>`) — bound globally on hover and released on leave, rather than left bound permanently, so scrolling this list doesn't accidentally also scroll some other window that happens to be open at the same time.

```
    for i, (question, answer) in enumerate(QUERIES, start=1):
        card = tk.Frame(scroll_frame, bg=CARD)
        card.pack(fill="x", pady=6, padx=2)
        accent = tk.Frame(card, bg=OPTION_COLORS[9], width=4)
        accent.pack(side="left", fill="y")
        body = tk.Frame(card, bg=CARD, padx=14, pady=10)
        body.pack(side="left", fill="both", expand=True)
        make_label(body, f"Q{i}.  {question}", 11, bold=True, ...).pack(anchor="w")
        tk.Label(body, text=f"→  {answer}", ...).pack(anchor="w", pady=(4, 0))
```

Builds one card per question inside `scroll_frame`: a thin colored accent strip on the left edge, then the question text in bold followed by the computed answer in cyan just below it — `enumerate(QUERIES, start=1)` numbers them `Q1`–`Q10` for display without needing to store the number in the tuple itself.

---

22. Entry Point

```
def main():
    global root
    load_gpa_scale()
    root = tk.Tk()
    root.title("FEST Summer 2026")
    root.configure(bg=BG)
    sw = root.winfo_screenwidth()
    sh = root.winfo_screenheight()
    root.geometry(f"{sw}x{sh}+0+0")
    root.resizable(True, True)
    show_welcome()
    root.mainloop()


if __name__ == "__main__":
    main()
```

The whole program's starting point. `global root` lets this function assign to the module-level `root` variable every other function in the file already refers to. `load_gpa_scale()` runs first so the editable grading scale is loaded from disk *before* any screen that might need it (e.g. a live GPA preview) can possibly open. `tk.Tk()` creates the one and only root window; `winfo_screenwidth()` / `winfo_screenheight()` read the monitor's resolution, and `root.geometry(f"{sw}x{sh}+0+0")` sizes and positions the window to exactly fill the screen

from the top-left corner — a manual full-screen effect rather than using a platform-specific "maximize" call. `show_welcome()` draws the very first screen, and `root.mainloop()` — the last line that runs — hands control over to Tkinter's event loop, which is what actually keeps the window open and responsive to clicks/keystrokes until it's closed; nothing after `root.mainloop()` in this function runs until the window is destroyed. `if __name__ == "__main__": main()` is the standard guard that only auto-runs the app when `main.py` is executed directly (`python main.py`), not if it were ever imported as a module from somewhere else.

---

23. Summary

`main.py` is a single-file Tkinter application with a strict separation between **data** (three flat files: `users.csv`, `students.csv`, `gpa_scale.json`, all read fresh and written immediately on every change — no in-memory database, no caching layer) and **presentation** (one root window that gets wiped and redrawn between top-level screens, plus modal `Toplevel` popups for each of the 10 numbered menu options). A handful of small conventions repeat throughout: grouping the flat student roster by `student_id` whenever a screen needs "one entry per student," using a single-element list (`[0]`) as a mutable box so nested functions can update shared state, and building custom rounded/colored buttons on a `Canvas` (later replaced by styled `ttk.Button`s specifically where native Tk buttons turned out to ignore color options on macOS). File I/O for the two CSVs is handled through **pandas** (`pd.read_csv` / `to_csv`) rather than the standard-library `csv` module, while charting (Options 4–6) optionally upgrades to **Seaborn** when it's installed and otherwise falls back to hand-written Matplotlib/NumPy equivalents.